

Prode UTN — Guía de Frontend

Especificación completa para construir una aplicación React elegante, moderna y profesional

- React + Vite
- TypeScript
- Tailwind + shadcn/ui
- JWT
- Contrato API verificado

Trabajo Práctico Integrador — Programación 4 — UTN FRVM
Backend: Spring Boot 3.4.12 · Java 21 · PostgreSQL
Contrato de API verificado contra el backend real (Etapas 1 a 6)

1. Objetivo

Construir el frontend de una plataforma de pronósticos deportivos (Prode) que se vea y se sienta como un **producto digital real**: elegante, moderno, minimalista y premium. No un formulario universitario básico. El backend ya está terminado y probado (Etapas 1 a 6); este documento define cómo construir el cliente que lo consume.

Contrato verificado. Todos los endpoints, bodies y respuestas de este documento fueron verificados directamente contra el código del backend y probados en local. Si seguís este contrato al pie de la letra, la integración funciona sin sorpresas.

2. Stack recomendado

Tecnología	Para qué	Por qué
Vite	Build tool y dev server	Arranque instantáneo, HMR rápido, configuración mínima.
React 18	Librería de UI	Estándar de la industria, enorme ecosistema.
TypeScript	Tipado estático	Detecta errores de contrato API antes de ejecutar. Curva baja si se tipa solo la API.
React Router v6	Navegación y rutas protegidas	Estándar para SPAs, soporta layouts anidados y guards.
Axios	Cliente HTTP	Interceptors para inyectar el JWT en todos los requests desde un solo lugar.
Tailwind CSS	Estilado	Consistencia visual garantizada, iteración rápida, sin CSS manual disperso.
shadcn/ui	Componentes base	Copia el código a tu repo: lo controlás y customizás. No impone su estética como MUI.
Lucide React	Íconos	Íconos limpios, modernos, livianos y consistentes.
Framer Motion	Animaciones (limitado)	Solo para entrada de cards y transiciones de página. No abusar.

¿TypeScript o JavaScript?

TypeScript. El mayor beneficio se obtiene tipando solo las respuestas de la API (un archivo `types/api.types.ts`). Eso da el 80% del valor con bajo costo de aprendizaje. Si el equipo nunca lo usó, pueden empezar permisivo y endurecer luego, pero no mezclar `.js` y `.tsx` sin criterio.

¿Qué NO usar

- **Redux / Zustand:** innecesario. Context API alcanza para auth y estado global de este TPI.
- **Material UI / Ant Design / Bootstrap:** identifican el proyecto como "genérico" y son difíciles de hacer ver premium.
- **Next.js:** es full-stack; acá el backend ya está en Spring Boot.
- **CSS-in-JS (styled-components):** más complejo que Tailwind para el mismo resultado.

3. Sistema de diseño

Identidad: **dark mode deportivo premium**, estilo app de estadísticas real (inspiración: Sofascore, FotMob, paneles de analytics). La elegancia viene de la *restricción*: pocos colores, mucho espacio, jerarquía clara.

Paleta de colores

```
// tailwind.config.js → theme.extend.colors
brand: {
  bg:      '#0B0F1A', // fondo principal (casi negro azulado)
  surface: '#131929', // fondo de cards
  border:  '#1E2D45', // bordes sutiles
  primary: '#3B82F6', // azul: botones, links, foco
  accent:  '#F59E0B', // ámbar: puntos, aciertos, destacados
  success: '#10B981', // verde: estado EN_JUEGO
  muted:   '#64748B', // texto secundario
  text:    '#E2E8F0', // texto principal
  danger:  '#EF4444', // errores, eliminar
}
```

Tipografía

Fuente única: **Inter** (Google Fonts). Pesos 400/500/600/700/800.

Nivel	Uso	Estilo
H1	Nombre app, landing	800, 2.5rem
H2	Títulos de sección	700, 1.5rem
H3	Títulos de card	600, 1.1rem
Body	Texto general	400, 0.95rem
Caption	Labels, metadatos	500, 0.8rem, color muted

Componentes visuales clave

Elemento	Estilo
Card de partido	Fondo <code>surface</code> , radius 12px, borde izquierdo de color según estado, hover con <code>translateY(-2px)</code> .
Badge de estado	POR_JUGARSE: slate. EN_JUEGO: verde con punto pulsante. FINALIZADO: gris.
Botón primario	<code>bg-blue-600 hover:bg-blue-700 text-white rounded-lg px-4 py-2</code>
Botón peligro	<code>bg-red-600 hover:bg-red-700</code> — solo para eliminar.
Animación	Entrada de cards con stagger (delay por índice), transición de página con fade. Nada más.

4. Variables de entorno

Nunca hardcodear la URL del backend. Crear `.env` en la raíz del frontend:

```
VITE_API_URL=http://localhost:8080
```

Y leerla siempre con `import.meta.env.VITE_API_URL`. Agregar `.env` al `.gitignore` del frontend.

CORS: el backend ya acepta peticiones desde `http://localhost:5173` (puerto por defecto de Vite). Si cambiás el puerto del dev server, avisá para ajustar el backend.

5. Autenticación y JWT

El backend devuelve un JWT al registrarse o loguearse. Ese token debe:

- Guardarse en `localStorage` (clave `token`).
- Enviarse en **todos** los requests protegidos como header `Authorization: Bearer <token>`.
- Borrarse al hacer logout o ante un `401`.

Seguridad — reglas no negociables:

- Nunca mostrar el token en pantalla, en la URL ni en `console.log`.
- Nunca guardar contraseñas (solo se envían en el request de login/registro).
- El `role` del usuario viene en la respuesta de login/registro — usarlo para mostrar/ocultar UI, pero recordar que la seguridad real la impone el backend.

6. Contrato de API completo

Verificado contra el backend real. Base URL: `http://localhost:8080` . Salvo Auth, todos requieren header `Authorization: Bearer <token>` .

Formato de error (todas las respuestas de error)

El backend siempre responde los errores con esta estructura uniforme. El campo a mostrar al usuario es `message` :

```
{
  "timestamp": "2026-06-09T12:30:18.037Z",
  "status": 400,
  "error": "Bad Request",
  "message": "El usuario ya pertenece a este grupo",
  "path": "/api/groups/join"
}
```

Enums del sistema

Enum	Valores
Role	USER , ADMIN
MatchStatus	POR_JUGARSE , EN_JUEGO , FINALIZADO
MatchDayStatus	PROGRAMADA , EN_JUEGO , FINALIZADA
ResultTrend	LOCAL , EMPATE , VISITANTE

6.1 — Auth público

POST `/api/auth/register` → 201 Created

```
// Request body
{ "username": "juan", "email": "juan@test.com", "password": "Password123" }
// password: mínimo 8 caracteres · email: formato válido

// Response 201
{ "token": "eyJ... ", "tokenType": "Bearer",
  "username": "juan", "email": "juan@test.com", "role": "USER" }

// Error 400 → { "message": "El email ya esta registrado" } (estructura completa de error)
```

POST `/api/auth/login` → 200 OK

OJO — campo clave: el login usa `usernameOrEmail` , no `username` . Acepta cualquiera de los dos como identificador.

```
// Request body
{ "usernameOrEmail": "juan", "password": "Password123" }

// Response 200
{ "token": "eyJ... ", "tokenType": "Bearer",
  "username": "juan", "email": "juan@test.com", "role": "USER" }

// Error 401 → { "message": "Credenciales invalidas" }
```

GET `/api/auth/me` autenticado → 200 OK

```
// Response
{ "id": 4, "username": "juan", "email": "juan@test.com",
  "role": "USER", "createdAt": "2026-06-09T12:00:00Z" }
```

6.2 — Teams / Equipos

Método	Endpoint	Rol	Notas
POST	/api/teams	ADMIN	Body { name } (max 100). 201.
GET	/api/teams	auth	Query opcional ?name= (filtro parcial).
GET	/api/teams/{id}	auth	Un equipo.
DELETE	/api/teams/{id}	ADMIN	204. Falla si tiene partidos.

```
// TeamResponse
{ "id": 1, "name": "River Plate", "createdAt": "..." }
```

No existe endpoint de edición (PUT) de equipos.

6.3 — MatchDays / Fechas (prefijo con guion: /api/match-days)

Método	Endpoint	Rol	Notas
POST	/api/match-days	ADMIN	Body { name } (max 120). 201. Nace PROGRAMADA.
GET	/api/match-days	auth	Query opcional ?status= (MatchDayStatus).
GET	/api/match-days/{id}	auth	Una fecha.
PUT	/api/match-days/{id}	ADMIN	Body { name } . Solo si PROGRAMADA y sin partidos.
DELETE	/api/match-days/{id}	ADMIN	204. Solo si PROGRAMADA y sin partidos.

```
// MatchDayResponse - el status lo calcula el backend, no se edita manualmente
{ "id": 1, "name": "Fecha 1", "status": "PROGRAMADA", "createdAt": "..." }
```

6.4 — Matches / Partidos

Método	Endpoint	Rol	Notas
POST	/api/matches	ADMIN	Crear. 201.
GET	/api/matches	auth	Query opcional ?matchDayId= .
GET	/api/matches/{id}	auth	Un partido.
PUT	/api/matches/{id}	ADMIN	Editar. Solo si POR_JUGARSE y sin pronósticos.
PATCH	/api/matches/{id}/start	ADMIN	POR_JUGARSE → EN_JUEGO.
PATCH	/api/matches/{id}/result	ADMIN	Carga resultado → FINALIZADO. Calcula puntos.
DELETE	/api/matches/{id}	ADMIN	204. Solo si POR_JUGARSE y sin pronósticos.

```
// POST body (crear)
{ "matchDayId": 1, "homeTeamId": 1, "awayTeamId": 2,
  "startTime": "2026-06-20T19:00:00Z" } // ISO-8601 UTC obligatorio

// PUT body (editar)
{ "homeTeamId": 1, "awayTeamId": 2, "startTime": "2026-06-20T19:00:00Z" }

// PATCH /result body
{ "homeGoals": 2, "awayGoals": 1 } // enteros ≥ 0; el backend calcula la tendencia

// MatchResponse (homeGoals/awayGoals/resultTrend son null hasta FINALIZADO)
{ "id": 1, "matchDayId": 1, "matchDayName": "Fecha 1",
  "homeTeamId": 1, "homeTeamName": "River Plate",
  "awayTeamId": 2, "awayTeamName": "Boca Juniors",
  "startTime": "2026-06-20T19:00:00Z", "status": "POR_JUGARSE",
  "homeGoals": null, "awayGoals": null, "resultTrend": null,
  "createdAt": " ... ", "updatedAt": " ... " }
```

6.5 — Predictions / Pronósticos (el matchId va en la URL, no en el body)

Método	Endpoint	Rol	Notas
POST	/api/predictions/matches/{matchId}	auth	Crea o actualiza (upsert) el pronóstico propio.
GET	/api/predictions/me	auth	Mis pronósticos. Query opcional <code>?matchStatus=</code> .
GET	/api/predictions/matches/{matchId}	auth	Pronósticos de un partido. Solo tras cerrar la carga.

```
// POST body – solo goles; la tendencia la calcula el backend
{ "predictedHomeGoals": 2, "predictedAwayGoals": 1 } // enteros ≥ 0

// PredictionResponse
{ "id": 1, "userId": 4, "username": "juan",
  "matchId": 1, "matchDayName": "Fecha 1",
  "homeTeamName": "River Plate", "awayTeamName": "Boca Juniors",
  "matchStartTime": " ... ", "matchStatus": "POR_JUGARSE",
  "predictedHomeGoals": 2, "predictedAwayGoals": 1, "predictedTrend": "LOCAL",
  "points": 0, "exactHit": false, "createdAt": " ... ", "updatedAt": " ... " }
```

Reglas que el frontend debe reflejar: solo se puede pronosticar un partido `POR_JUGARSE` y hasta 30 minutos antes del inicio. Pasado ese punto, deshabilitar el formulario y mostrar el motivo. Los `points` y `exactHit` quedan en 0/false hasta que el ADMIN carga el resultado.

6.6 — Rankings

GET /api/rankings/global autenticado

```
// RankingResponse[] – orden: puntos desc, exactHits desc, username asc
[ { "position": 1, "userId": 4, "username": "juan",
  "totalPoints": 7, "exactHits": 2, "predictionsCount": 4 } ]
```

6.7 — Groups / Grupos privados

Etapa 6

Método	Endpoint	Acceso	Notas
POST	/api/groups	auth	Crear grupo. 201. El creador queda como owner y miembro.
POST	/api/groups/join	auth	Unirse por código de invitación.
GET	/api/groups/me	auth	Mis grupos. Lista vacía si ninguno.
GET	/api/groups/{groupId}	miembro	Detalle con lista de miembros.
DELETE	/api/groups/{groupId}/members/me	miembro	Salir del grupo. 204. El owner no puede salir.
GET	/api/groups/{groupId}/ranking	miembro	Ranking solo de los miembros del grupo.

```
// POST /api/groups body
{ "name": "Amigos UTN" } // max 100, obligatorio

// POST /api/groups/join body
{ "inviteCode": "493B33CB" } // max 50, obligatorio

// GroupResponse (create / join / me)
{ "id": 1, "name": "Amigos UTN", "inviteCode": "493B33CB",
  "ownerId": 4, "ownerUsername": "juan", "membersCount": 2, "createdAt": "..." }

// GroupDetailResponse (GET /{groupId})
{ "id": 1, "name": "Amigos UTN", "inviteCode": "493B33CB",
  "ownerId": 4, "ownerUsername": "juan", "membersCount": 2,
  "members": [
    { "userId": 4, "username": "juan", "joinedAt": "..." },
    { "userId": 5, "username": "benja", "joinedAt": "..." } ],
  "createdAt": "..." }

// GET /{groupId}/ranking → RankingResponse[] (mismo shape que ranking global)
```

Errores propios de grupos (todos con la estructura de error estándar, campo `message`):

Situación	HTTP	message
Código inexistente	404	No existe un grupo con el codigo de invitacion indicado
Ya es miembro	400	El usuario ya pertenece a este grupo
No miembro accede a detalle	400	No tenes permisos para acceder a este grupo
No miembro ve ranking	400	No tenes permisos para ver el ranking de este grupo
Owner intenta salir	400	El creador del grupo no puede abandonar el grupo

7. Estructura de carpetas

```
src/
├─ api/
│  └─ apiClient.ts      // axios con baseURL + interceptors (JWT, 401)
│  └─ endpoints.ts      // funciones por recurso (auth, teams, matches, groups...)
├─ auth/
│  └─ AuthContext.tsx   // user, token, role, login(), logout()
│  └─ authService.ts    // llamadas register/login
│  └─ ProtectedRoute.tsx // guard por autenticación y rol
├─ components/
│  └─ layout/ (AppLayout, AdminLayout, Navbar, Sidebar)
│  └─ ui/ (Button, Card, Badge, Spinner, EmptyState)
│  └─ feedback/ (ErrorMessage, Toast, ConfirmDialog)
│  └─ match/ (MatchCard, MatchList, MatchResultForm)
│  └─ group/ (GroupCard, JoinGroupForm, MembersList)
│  └─ ranking/ (RankingTable, RankingRow)
├─ hooks/ (useAuth, useMatches, usePredictions, useRanking, useGroups)
├─ pages/
│  └─ LandingPage, LoginPage, RegisterPage
│  └─ UserDashboardPage, MatchesPage, PredictionsPage, RankingPage
│  └─ GroupsPage, GroupDetailPage
│  └─ admin/ (AdminDashboard, TeamsPage, MatchDaysPage, MatchesAdminPage)
│  └─ NotFoundPage
├─ types/ api.types.ts // interfaces de TODAS las respuestas
├─ utils/ formatDate.ts, formatStatus.ts, errorHandler.ts
├─ styles/ globals.css
├─ App.tsx // router
└─ main.tsx
```

8. Archivos clave (código de arranque)

api/apiClient.ts

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: { 'Content-Type': 'application/json' },
});

// Inyecta el JWT en TODOS los requests
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Logout automático ante 401
apiClient.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

utils/errorHandler.ts

```
import axios from 'axios';
// El backend SIEMPRE manda { message } dentro del body de error.
export function extractErrorMessage(error: unknown, fallback: string): string {
  if (axios.isAxiosError(error)) {
    return error.response?.data?.message ?? fallback;
  }
  return fallback;
}
```

auth/ProtectedRoute.tsx

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../AuthContext';

export function ProtectedRoute({ requiredRole }: { requiredRole?: 'ADMIN' }) {
  const { isAuthenticated, isAdmin } = useAuth();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (requiredRole === 'ADMIN' && !isAdmin) return <Navigate to="/dashboard" replace />;
  return <Outlet />;
}
```

types/api.types.ts (extracto)

```
export type Role = 'USER' | 'ADMIN';
export type MatchStatus = 'POR_JUGARSE' | 'EN_JUEGO' | 'FINALIZADO';
export type ResultTrend = 'LOCAL' | 'EMPATE' | 'VISITANTE';

export interface AuthResponse {
  token: string; tokenType: string;
  username: string; email: string; role: Role;
}

export interface Match {
  id: number; matchDayId: number; matchDayName: string;
  homeTeamId: number; homeTeamName: string;
  awayTeamId: number; awayTeamName: string;
  startTime: string; status: MatchStatus;
  homeGoals: number | null; awayGoals: number | null;
  resultTrend: ResultTrend | null;
}

export interface GroupResponse {
  id: number; name: string; inviteCode: string;
  ownerId: number; ownerUsername: string;
  membersCount: number; createdAt: string;
}

export interface RankingEntry {
  position: number; userId: number; username: string;
  totalPoints: number; exactHits: number; predictionsCount: number;
}
```

9. Pantallas y prioridad para la demo

Prioridad	Pantalla	Por qué
ALTA	Login / Registro	Punto de entrada. Tiene que verse impecable.
ALTA	Lista de partidos (MatchCard)	Corazón visual. Cards con badge de estado por color.
ALTA	Formulario de pronóstico	Contadores +/- de goles, tendencia calculada en vivo.
ALTA	Ranking global	Tabla premium, top 3 destacado, fila propia resaltada.
ALTA	Panel admin (start + result)	Demuestra el ciclo completo del partido.
MEDIA	Grupos (crear, unirse, detalle, ranking)	Diferencial de Etapa 6. Muy vistoso para la defensa.
MEDIA	Dashboard usuario	Resumen: mis puntos, posición, próximos partidos.
BAJA	Landing, Perfil	Si sobra tiempo.

Componentes a construir PRIMERO

Antes de cualquier página, crear los componentes base y reutilizarlos en todos lados. La consistencia visual es lo que hace ver profesional:

Badge Button Card Spinner EmptyState ErrorMessage ConfirmDialog MatchCard

10. Flujo de demo recomendado

Este orden demuestra TODO el sistema en vivo (auth, roles, CRUD, ciclo de partido, scoring automático, ranking global y por grupo):

1. Abrir landing → mostrar estética general.
2. Registrar `usuario_demo` y `usuario_demo2`.
3. Login como **ADMIN** → mostrar panel diferenciado.
4. ADMIN crea 2 equipos (River, Boca).
5. ADMIN crea una fecha ("Fecha Demo").
6. ADMIN crea un partido River vs Boca (inicio en ~1 hora).
7. Login `usuario_demo` → ver el partido → pronosticar **2-1**.
8. Login `usuario_demo2` → pronosticar **0-0**.
9. `usuario_demo` crea un grupo → copia el `inviteCode`.
10. `usuario_demo2` se une al grupo con ese código.
11. Login ADMIN → **iniciar** el partido (badge pasa a verde EN_JUEGO).
12. ADMIN **carga resultado 2-1** → partido FINALIZADO, puntos calculados.
13. Login `usuario_demo` → "Mis pronósticos" muestra **3 puntos** (acierto exacto).
14. Ver **ranking global** → demo lidera.
15. Ver **ranking del grupo** → solo los 2 miembros del grupo.

11. Manejo de errores y estados

Errores del backend

Siempre extraer `error.response.data.message` con `extractErrorMessage()` y mostrarlo en un `<ErrorMessage>` lindo. Nunca mostrar el error crudo de Axios ni stacktraces.

Loading states

Cada fetch con spinner o skeleton. Nunca una pantalla en blanco. Para listas, skeleton de cards (mejor que spinner).

Empty states (con ícono de Lucide)

- Sin partidos: "Todavía no hay partidos. El admin los agregará pronto."
- Sin pronósticos: "Aún no hiciste ningún pronóstico. ¡Aprovechá antes de que empiecen!"
- Sin grupos: "No estás en ningún grupo. Creá uno o unite con un código."
- Ranking vacío: "Nadie tiene puntos aún. ¡Sé el primero!"

12. Qué evitar

No hacer	Hacer en su lugar
Hardcodear URLs en componentes	Centralizar en <code>endpoints.ts</code> con <code>VITE_API_URL</code>
Guardar el token en varios lugares	Solo localStorage, vía AuthContext
Mostrar token en pantalla / console.log	Nunca exponerlo
Fetch directo en componentes	Hooks propios (useMatches, useGroups...)
Usar <code>any</code> en TypeScript	Tipar con las interfaces de <code>api.types.ts</code>
Mostrar el error crudo de Axios	Siempre <code>extractErrorMessage()</code>
Olvidar loading y empty states	Cada lista los tiene
console.log en la entrega	Limpiar antes de la defensa

13. Checklist previo a la defensa

- ☐ Login y registro funcionan contra la API real (recordar: login usa `usernameOrEmail`).
- ☐ El JWT se envía en cada request protegido.
- ☐ Un USER no puede entrar al panel admin (ProtectedRoute con rol).
- ☐ El badge de estado del partido cambia con el estado real.
- ☐ Al cargar resultado, los puntos se reflejan en "Mis pronósticos" y en el ranking.
- ☐ Crear grupo, unirse, ver detalle y ranking del grupo funcionan.
- ☐ Todos los formularios muestran los errores del backend con estilo.
- ☐ Todas las listas tienen empty state y loading state.
- ☐ La app es responsive en móvil.
- ☐ Sin `console.log` ni tokens visibles.